

VLSI DESIGN INTERNSHIP – TASK 2

Verilog HDL Codes and Testbench Programs

Submitted By:

Sowjanya Gollapalli

Internship Duration:

May 20, 2026 – July 5, 2026

Tools Used:

- Xilinx Vivado

- CircuitVerse

1. AND Gate

Verilog Code:

```
module and_gate(  
    input A,  
    input B,  
    output Y  
);  
    assign Y = A & B;  
endmodule
```

2. OR Gate

Verilog Code:

```
module or_gate(  
    input A,  
    input B,  
    output Y  
);  
    assign Y = A | B;  
endmodule  
---
```

3. NOT Gate

Verilog Code:

```
module not_gate(  
    input A,  
    output Y  
);  
    assign Y = ~A;  
endmodule  
---
```

4. NAND Gate

Verilog Code:

```
module nand_gate(  
    input A,  
    input B,  
    output Y
```

```
);  
assign Y = ~(A & B);  
endmodule  
---
```

5. NOR Gate

Verilog Code:

```
module nor_gate(  
    input A,  
    input B,  
    output Y  
);  
assign Y = ~(A | B);  
endmodule  
---
```

6. XOR Gate

Verilog Code:

```
module xor_gate(  
    input A,  
    input B,  
    output Y  
);  
assign Y = A ^ B;  
endmodule
```

7. Half Adder

Verilog Code:

```
module half_adder(  
    input A,  
    input B,  
    output SUM,  
    output CARRY  
);  
    assign SUM = A ^ B;  
    assign CARRY = A & B;  
endmodule
```

Half Adder Testbench:

```
`timescale 1ns / 1ps  
module half_adder_tb;  
    reg A;  
    reg B;  
    wire SUM;  
    wire CARRY;  
    half_adder uut(  
        .A(A),  
        .B(B),  
        .SUM(SUM),
```

```
.CARRY(CARRY)
```

```
);
```

```
initial
```

```
begin
```

```
A=0; B=0;
```

```
#10;
```

```
A=0; B=1;
```

```
#10;
```

```
A=1; B=0;
```

```
#10;
```

```
A=1; B=1;
```

```
#10;
```

```
$finish;
```

```
end
```

```
endmodule
```

```
---
```

8. Full Addder

Verilog Code:

```
module full_adder(
```

```
input A,
```

```
input B,
```

```
input Cin,
```

```
output SUM,
```

```
output CARRY
```

```
);  
assign SUM = A ^ B ^ Cin;  
assign CARRY =  
(A & B) |  
(B & Cin) |  
(A & Cin);  
endmodule  
---
```

Full Adder Testbench:

```
`timescale 1ns / 1ps  
module full_adder_tb;  
reg A;  
reg B;  
reg Cin;  
wire SUM;  
wire CARRY;  
full_adder uut(  
    .A(A),  
    .B(B),  
    .Cin(Cin),  
    .SUM(SUM),  
    .CARRY(CARRY)  
);
```

```
initial
begin
A=0; B=0; Cin=0;
#10;
A=0; B=0; Cin=1;
#10;
A=0; B=1; Cin=0;
#10;
A=0; B=1; Cin=1;
#10;
A=1; B=0; Cin=0;
#10;
A=1; B=0; Cin=1;
#10;
A=1; B=1; Cin=0;
#10;
A=1; B=1; Cin=1;
#10;
$finish;
end
endmodule
---
```

Expected Simulation Results

Half Adder:

A| B| SUM| CARRY

0| 0| 0| 0

0| 1| 1| 0

1| 0| 1| 0

1| 1| 0| 1

Full Adder:

A| B| Cin| SUM| CARRY

0| 0| 0| 0| 0

0| 0| 1| 1| 0

0| 1| 0| 1| 0

0| 1| 1| 0| 1

1| 0| 0| 1| 0

1| 0| 1| 0| 1

1| 1| 0| 0| 1

1| 1| 1| 1| 1

Conclusion:

Successfully implemented AND, OR, NOT, NAND, NOR, XOR, Half Adder, and Full Adder circuits using Verilog HDL. Functional verification was performed using dedicated testbenches and simulation waveforms in Xilinx Vivado.